

LLD – Storage Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Storage module of OpenScholar. The module is responsible for handling file uploads (PDFs), storage management, retrieval, and ensuring compatibility with both local object storage (MinIO) and cloud storage (AWS S3).

The design prioritizes reliability, scalability, and seamless migration from VPS-based storage to cloud infrastructure.

2. Scope

The Storage module is responsible for:

- Upload PDF files
- Store files in object storage (MinIO → S3)
- Generate file URLs for access
- Manage file naming and versioning
- Handle upload failures and retries

Out of Scope:

- CDN delivery optimization (future)
 - File compression
-

3. Storage Architecture

Phase 1 (MVP)

- MinIO running on VPS
- S3-compatible API

Phase 2 (Scaling)

- AWS S3
 - Optional CDN (CloudFront)
-

4. File Naming Strategy

Format:

papers/{paperId}/v{versionNumber}.pdf

Example:

papers/123e4567-e89b-12d3-a456-426614174000/v1.pdf

Rules:

- Each version has unique file
- Overwrites are not allowed

5. API Contract (Internal Service)

uploadFile(file, paperId, versionNumber)

Input:

- file (binary)
- paperId (UUID)
- versionNumber (INT)

Output:

- fileUrl (string)

6. Upload Flow

Steps:

1. Receive file via API (multipart/form-data)
2. Validate file:
3. Type: application/pdf
4. Size $\leq$ 50MB
5. Generate file path using naming strategy
6. Upload file to storage (MinIO)
7. Receive file URL
8. Store URL in database (paper_versions.pdf_url)

7. Storage Service Logic

7.1 uploadFile()

Steps: 1. Validate file 2. Generate object key 3. Upload to MinIO/S3 4. Return public or signed URL

7.2 deleteFile() (optional future)

Steps: 1. Locate file by key 2. Delete from storage

8. Failure Handling

Upload Failure

- If upload fails:
- Do not save DB record
- Return error

Partial Failure (DB success, storage fail)

- Use transaction-like handling:
 - Upload first → then DB write
-

9. Security Considerations

- Validate file type strictly (PDF only)
- Limit file size ($\leq 50\text{MB}$)
- Prevent path traversal
- Use private buckets (recommended)

Future:

- Use signed URLs for secure access
-

10. Access Strategy

MVP

- Public URL access (simpler)

Future

- Signed URL (temporary access)
-

11. Prisma Integration

Field used:

paper_versions.pdf_url (TEXT)

12. Folder Structure

src/ lib/storage.ts

modules/storage/service.ts

13. Implementation Notes

- Use AWS SDK (S3 client) for both MinIO and S3
 - Keep storage logic isolated in lib/storage.ts
 - Do not mix storage logic with business logic
-

14. Acceptance Criteria

- System uploads PDF successfully
 - File is stored with correct naming
 - URL is saved in DB
 - File can be accessed via URL
 - Upload failures handled correctly
-

15. Future Enhancements

- Direct upload via presigned URLs
 - CDN integration
 - File version cleanup
 - Virus scanning
-

End of Document